

Pemrograman Berorientasi Objek



MVC [2]

Outline

- ▶ MVC Implementation: Java Web
 - Java Server Pages (JSP)
 - Servlet

Request in web app

- In web app, URL address can be a view (webpage: HTML, XML, or JSP) or a controller (servlet).
- Accessing a URL address means we are sending a request using a **method** to that view or controller.
- There are four methods available: GET, POST, PUT, and DELETE
- When method is not set, by default the method will be set to GET. For example:
 - when we open a view or a controller by typing manually the address in browser address bar, or
 - when we click the link (<a> tag) from inside a webpage

Request in web app

- ▶ Another way for sending a request is by submitting a form page. We can set the method (only GET or POST) as the value of *method* attribute in `<form>` tag

```
<form method="post" action="process.jsp">
```

```
...
```

```
</form>
```

- ▶ GET method

- This will be set automatically when *method* attribute in `<form>` tag is blank, not exist, or has value other than "POST".
- After form is submitted, form data is appended in the URL address

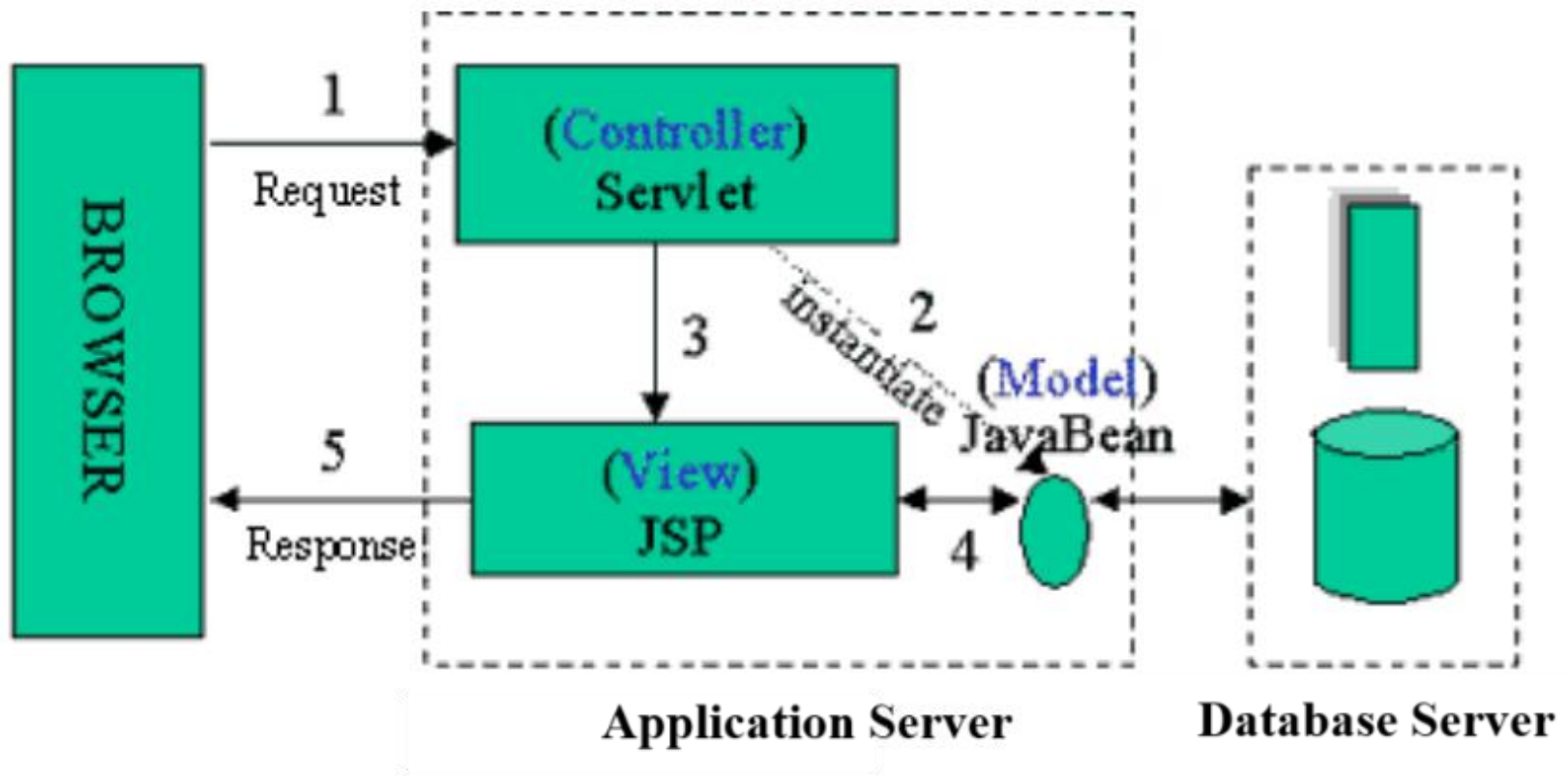
- ▶ POST method

- After form is submitted, form data is stored as request body, not seen in URL address

When to use GET or POST in form page

- ▶ If data is confidential or for manipulation (adding, editing), use POST
 - So the submitted data doesn't displayed in the URL address
 - Example: login, registration, data input, data update
- ▶ If submitted data is large, use POST
 - So the URL address doesn't become too long
 - Example: lots of fields/inputs or form has file attachment upload)
- ▶ Other than above, use GET (example: searching page, go to page)

Recall: MVC Architecture in Java Web



Java Server Pages

- ▶ Combination of HTML / XML and Java code in one file that is executed as a Java code
- ▶ Allows separation of view and logic, making development faster, easier, and reusable
- ▶ Application server will interpret the JSP file and the result will be rendered and displayed to the browser

Java Server Pages

► Try this code:

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>Hello world</title>
    </head>
    <body>
        <h1>Hello World!</h1>
        <%
            out.println("<br /> Your IP address is " + request.getRemoteAddr());
            String browser = request.getHeader("user-agent");
            out.print("<br /> and your browser is ");
            out.println(browser);
        %>
    </body>
</html>
```


Java Scripting in JSP

- ▶ Scriptlet, for general use
 - `<% ... %>`
- ▶ Expression, for functions or code that has result
 - `<%= ... %>`
 - Example:

```
<%@page import="java.util.Date"%>
Server date and time: <%=new Date()%>
```
- ▶ Declaration, for creating variables only
 - `<%! ... %>`
 - Example:

```
<%! int k = 0; %>
<%! static int k = 0; %>
```

Handle request data in JSP URL

- ▶ You can add request data in JSP. It is usually after the URL formed as *key=value* pair after 'somepage.jsp?'. For example:
<http://test.com/book.jsp?menu=view&sort=date>
- ▶ This means there are two data in this book.jsp page, *menu* key that contains "view" string and *sort* key that contains "date" string
- ▶ In JSP, you can get the parameter data with:
 - `request.getParameter("key_name")` (used inside scriptlet or expression)
 - `${param.key_name}` (directly in page)
- ▶ "key_name" is the key in *key=value* pair in URL address

JSP Action Tag

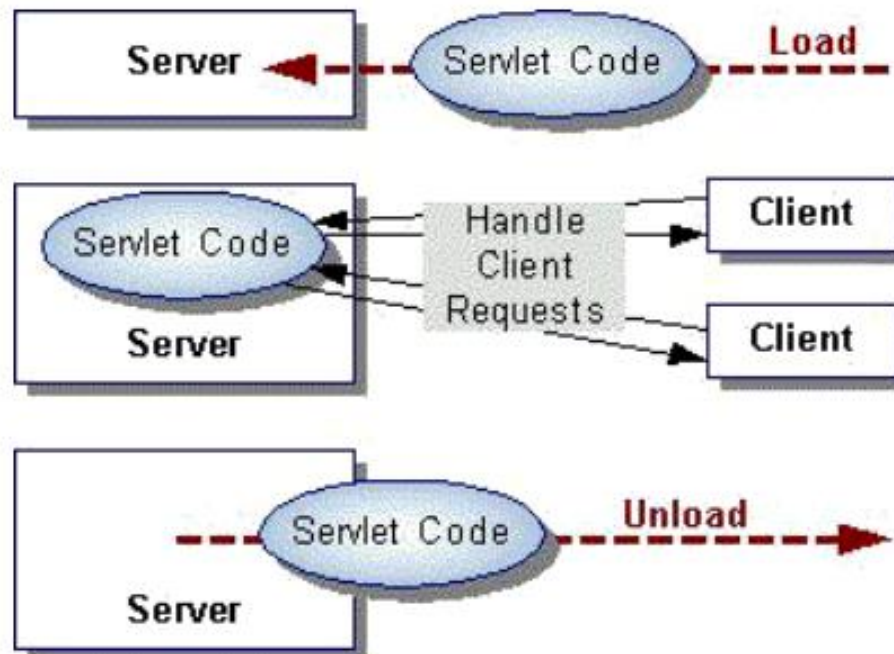
- ▶ Java provides a tag used within JSP called Action tag
- ▶ JSP Action tag is used to simplify some scriptlet code in your JSP
- ▶ There are many JSP action tags or elements, and each of them has its own uses and characteristics. Each JSP action tag is implemented to perform some precise tasks
- ▶ For example, `<jsp:include>` is widely used for creating pages with templates
- ▶ See <https://www.w3schools.in/jsp/actions> for details

Question?



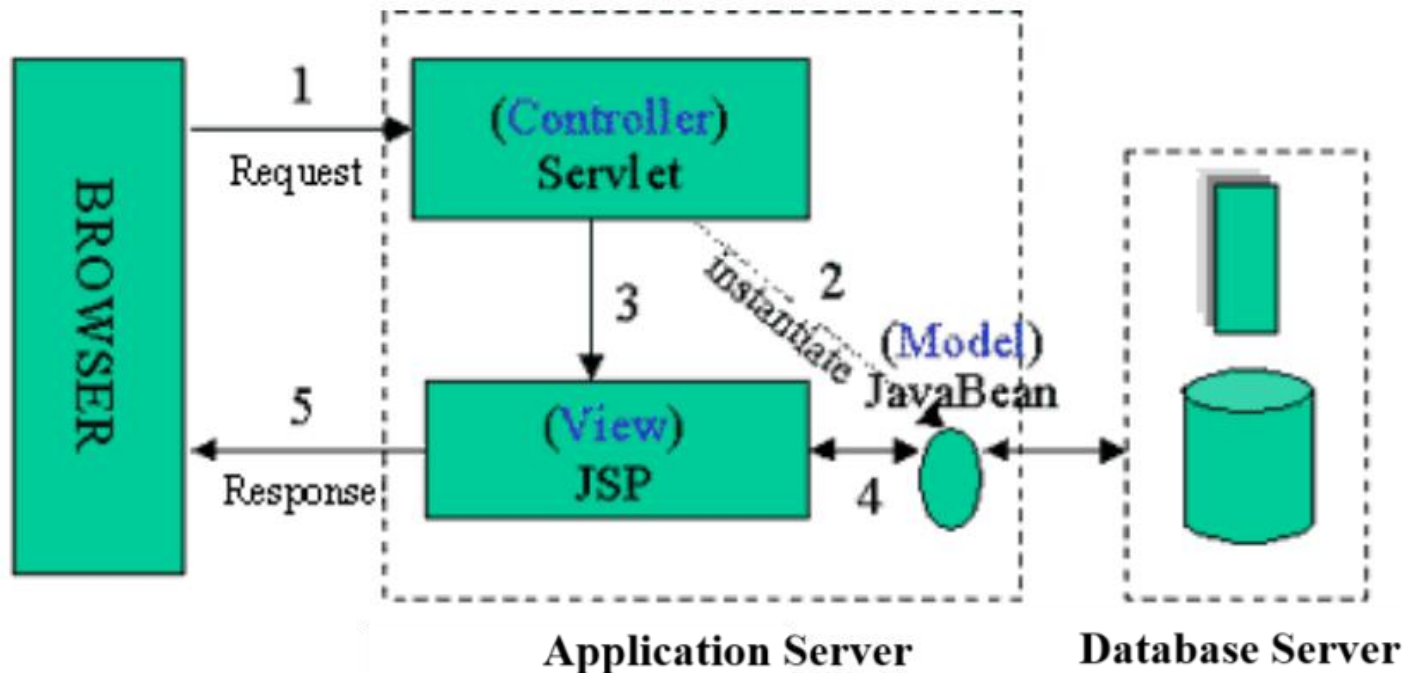
Servlet

- ▶ Java class that dynamically loaded for request & response processing



Recall: MVC in Java Web

- ▶ Servlet acts as the controller



Servlet

► Features:

- Dynamically process requests and construct responses
- Have access to the entire family of Java APIs
- Understand HTTP (able to access library of HTTP-specific calls)

► Classes:

- HttpServlet
- HttpServletRequest
- HttpServletResponse

Handle request data in servlet URL

- ▶ Like JSP, you can add data in servlet URL. It is usually formed as *key=value* pair after `/someservlet?`, or as segments (trailing slash, `/a/b`; mostly used in frameworks). For example: <http://test.com/book?menu=view&sort=date>
- ▶ This means there are two data in this servlet, *menu* key that contains "view" string and *sort* key that contains "date" string
- ▶ Same as JSP, you can get the request data with:
`request.getParameter("key_name")`
- ▶ "key_name" is the key in *key=value* pair in servlet address

Handle request data in form page

- ▶ You can get the submitted data with the same way:
`request.getParameter("attr_name")`
- ▶ “attr_name” is the value of your *name* attribute in input tag

```
<input type="text" name="attr_name" />
```

Servlet

► Methods:

- **init()** – the `init()` function is called when the servlet is initialized by the server. This often happens on the first `doGet()` or `doPost()` call of the servlet.
- **destroy()** – this function is called when the servlet is being destroyed by the server, typically when the server process is being stopped.
- **doGet()** – the `doGet()` function is called when the servlet is called via GET method.
- **doPost()** – the `doPost()` function is called when the servlet is called via POST method.

Passing data from Controller to View

- ▶ Common practice in MVC, we will never access a view (somepage.jsp) directly. All access will be via a controller (servlet).
- ▶ In Server-Side Rendering, a servlet processes some data, load a JSP file, and the data will be passed to that JSP. Then we can use that data inside the JSP
- ▶ Data is passed with a *name*, and will be accessed via that *name*

Example: Passing a string from Controller to View

```
//some servlet
```

```
request.setAttribute("keyword", "test");
```

```
// 'keyword' is the name, 'test' is the data
```

```
request.getRequestDispatcher("form_search.jsp").  
forward( request, response);
```

```
<!-- form_search.jsp -->
```

```
<%= request.getAttribute("keyword") %> or  
${keyword}
```

```
<!-- output: test -->
```

Example: JSP

► form_login.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Login Page</title>
  </head>
  <body>
    <h1>Login</h1>
    <form method="post" action="auth">
      Username: <input type="text" name="user" /><br />
      Password: <input type="password" name="pass" /><br />
      <input type="submit" value="Login" />
    </form>
    ${msg}
  </body>
</html>
```

Example: JSP

► form_search.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Search Page</title>
  </head>
  <body>
    <h1>Hello, user</h1>
    <a href="auth?logout"><button>Logout</button></a>
    <br /><br />
    <form method="get" action="home">
      Keyword : <input type="text" name="key" /><br />
      <input type="submit" value="Search" />
    </form>
    ${keyword}
  </body>
</html>
```

Example: Auth Servlet

```
@WebServlet(name = "Auth", urlPatterns = {"/auth"})
public class AuthController extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        request.getRequestDispatcher(path: "form_login.jsp").forward(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String username = request.getParameter(name: "user");
        String password = request.getParameter(name: "pass");
        if ("admin".equals(anObject: username) && "admin123".equals(anObject: password)) {
            response.sendRedirect(location: "home");
            return;
        }
        request.setAttribute(name: "msg", o: "username / password salah");
        request.getRequestDispatcher(path: "form_login.jsp").forward(request, response);
    }
}
```

Example: Home Servlet

```
@WebServlet(name = "Home", urlPatterns = {"/home"})  
public class HomeController extends HttpServlet {  
  
    @Override  
    protected void doGet(HttpServletRequest request, HttpServletResponse response)  
        throws ServletException, IOException {  
        String keyword = request.getParameter(name: "key");  
        request.setAttribute(name: "keyword", o: keyword);  
        request.getRequestDispatcher(path: "form_search.jsp").forward(request, response);  
    }  
}
```


How we differentiate the action for controller?

- ▶ Add additional request data in controller URL to differentiate the action in one servlet, such as for viewing, adding, and editing data

Example: Product Servlet

```
@WebServlet(urlPatterns = {"/product"})
public class ProductController extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String menu = request.getParameter(name: "action");
        if (menu == null) { //view menu
            //get all data from database
            request.getRequestDispatcher(path: "view_product.jsp").forward(request, response);
        } else if ("add".equals(anObject: menu)) {
            request.getRequestDispatcher(path: "form_product.jsp").forward(request, response);
        } else if ("edit".equals(anObject: menu)) {
            //get one data from database
            request.getRequestDispatcher(path: "form_product.jsp").forward(request, response);
        }
    }
}
```

Passing data besides string from Controller to View

- ▶ Same as passing a String data in servlet, but in view (JSP) you must do narrowing casting, and if it's a custom class you must import the class

```
//some servlet
```

```
Product p = new Product();
```

```
request.setAttribute("product", p);
```

```
request.getRequestDispatcher("form_product.jsp").forward( request, response);
```

```
//form_product.jsp
```

```
<%@page import="models.Product"%>
```

```
<% Product p = (Product) request.getAttribute("product"); %>
```

Example: Product model

```
package models;

public class Product {
    private int id;
    private String name;
    private double price;

    public Product(int id, String name, double price) {...5 lines }

    public int getId() {...3 lines }

    public void setId(int id) {...3 lines }

    public String getName() {...3 lines }

    public void setName(String name) {...3 lines }

    public double getPrice() {...3 lines }

    public void setPrice(double price) {...3 lines }
}
```

Example: Product servlet

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    String menu = request.getParameter(name: "action");
    if (menu == null) { //view menu
        request.setAttribute(name: "title", o: "Daftar Produk");
        ArrayList<Product> prods = new ArrayList<>();
        prods.add(new Product(id: 1, nama: "Nasi Goreng", harga: 15000));
        prods.add(new Product(id: 2, nama: "Nasi Mawud", harga: 20000));
        request.setAttribute(name: "list", o: prods);
        request.getRequestDispatcher(path: "view_product.jsp").forward(request, response);
    }
}
```

Example: view_product.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="java.util.ArrayList, models.Product" %>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>${title}</title>
  </head>
  <body>
    <h3>${title}</h3>
    <table border="1">
      <tr>
        <th>No</th>
        <th>Nama</th>
        <th>Harga</th>
      </tr>
      <%
        ArrayList<Product> prods = (ArrayList<Product>) request.getAttribute("list");
        int i = 1;
        for (Product p : prods) {
      %>
      <tr>
        <th><%= i++ %></th>
        <th><%= p.getName() %></th>
        <th><%= p.getPrice() %></th>
      </tr>
      <% } %>
    </table>
  </body>
</html>
```

Passing object data from Controller to View

- ▶ If the object data is a single object, alternatively you can use JSP Action tag `<jsp:useBean>`. This tag will do narrowing casting automatically
- ▶ Product servlet:

```
} else if ("edit".equals(anObject: menu)) {  
    request.setAttribute(name: "title", o: "Edit Produk");  
    Product p = new Product(id: 2, nama: "Nasi Goreng", harga: 15000);  
    request.setAttribute(name: "product", o: p);  
    request.getRequestDispatcher(path: "form_product.jsp").forward(request, response);  
}
```

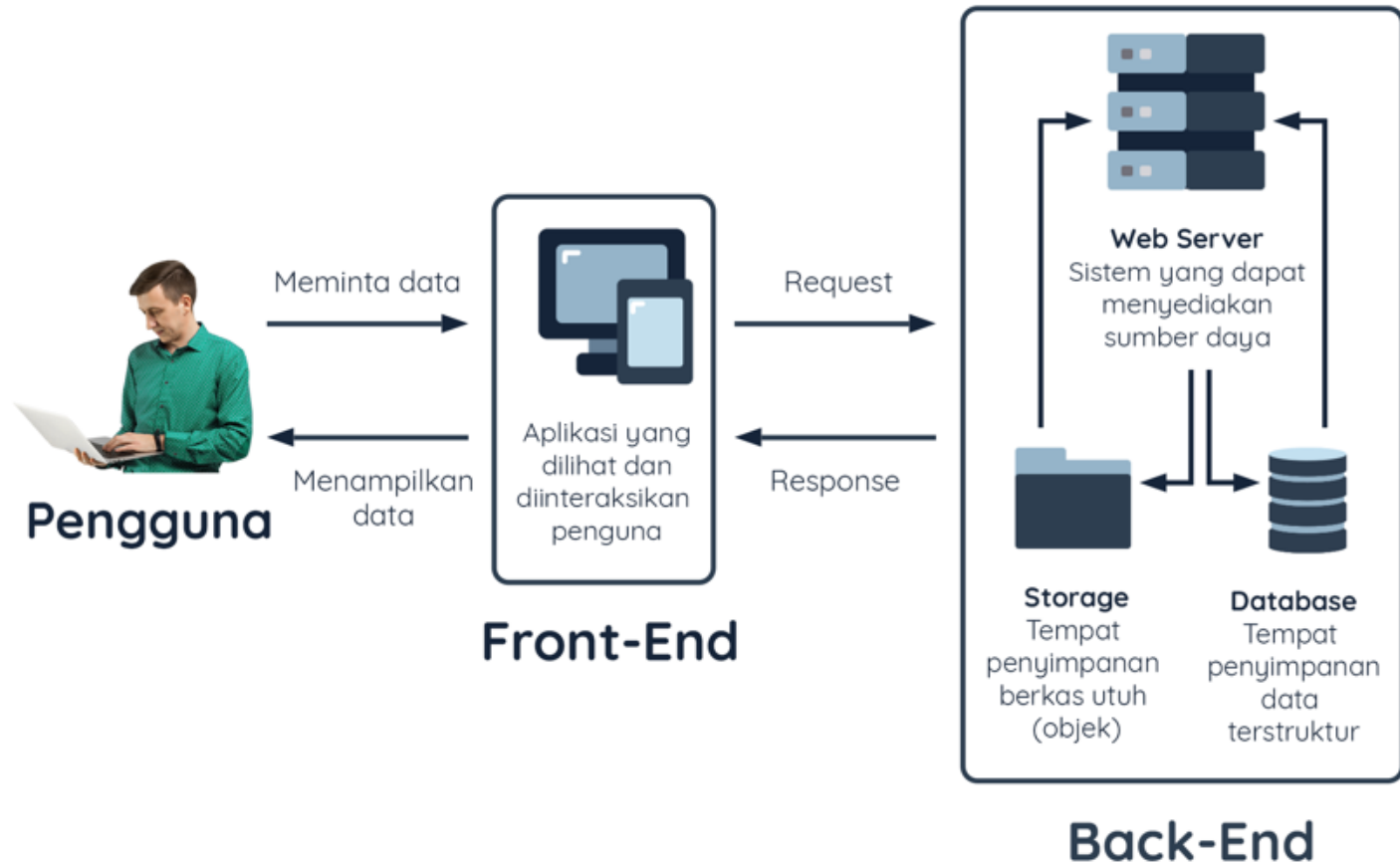
Example: form_product.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>${title}</title>
  </head>
  <body>
    <h3>${title}</h3>
    <jsp:useBean id="product" scope="request" type="models.Product"></jsp:useBean>
    <form method="post" action="product?action=u">
      Nama &nbsp;   : <input type="text" name="nama"
                        value="<jsp:getProperty name="product" property="name" />" />
      <br />
      Harga &nbsp;   : <input type="number" name="harga"
                        value="<jsp:getProperty name="product" property="price" />" />
      <br /><br />
      <input type="submit" value="Simpan" />
    </form>
  </body>
</html>
```


Passing data from Controller to View

- ▶ In Client-Side Rendering or API-driven, a servlet processes some data, formats the data as needed, and then returns that formatted data
- ▶ Data is processed in frontend or another backend
- ▶ How this works?

How back-end and front-end “talk”?



How front-end send request in CSR?

- ▶ In Server-Side Rendering, we can send request to back-end by going to another page, with a link or a form
- ▶ In Client-Side Rendering, we must use AJAX (Asynchronous Javascript & XML) to send request to back-end
- ▶ We can perform AJAX operation by using Javascript *fetch* function or external library (jQuery, Axios, etc)

Example: AJAX with fetch

- ▶ https://www.w3schools.com/js/js_api_fetch.asp

```
const options = {  
  method: 'POST',  
  headers: {  
    'Content-Type': 'application/json',  
  },  
  body: {  
    name: 'Nasi Bakar',  
    price: 25000,  
  },  
}  
  
fetch('http://localhost:8080/products/add', options);
```

Example: AJAX with jQuery

- ▶ <https://api.jquery.com/jquery.ajax/>

```
$.ajax({  
  method: "POST",  
  url: "some.php",  
  data: { name: "John", location: "Boston" }  
})  
  .done(function( msg ) {  
    alert( "Data Saved: " + msg );  
  });
```

Formatted data

- ▶ When Servlet returns data, we can send many data formats:
 - Plain text, Binary, HTML, JSON (JavaScript Object Notation)
- ▶ JSON format is often used for request and response in CSR
- ▶ JSON is structured like JavaScript objects, written as key/value pairs but keys are enclosed in double quotes ("). It can contain primitive values, objects, or arrays

```
{  
  "message": "Berikut daftar kopi yang tersedia",  
  "coffees": [  
    {  
      "id": 1,  
      "name": "Kopi Tubruk",  
      "price": 12000  
    },  
    {  
      "id": 2,  
      "name": "Kopi Tarik",  
      "price": 15000  
    },  
    {  
      "id": 3,  
      "name": "Kopi Jawa",  
      "price": 18000  
    }  
  ]  
}
```

Example: Product servlet

- ▶ Servlet returns JSON format, using GSON library

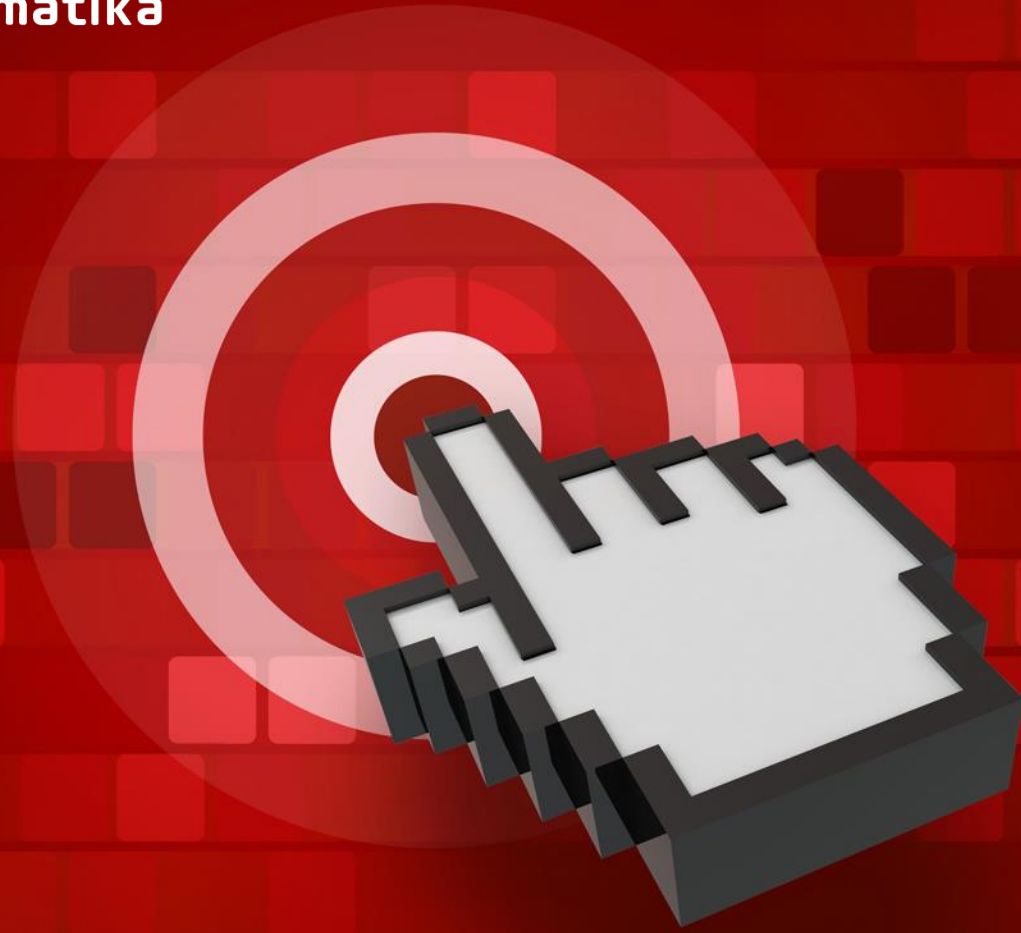
```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    String menu = request.getParameter("menu");
    if (menu == null) { //view product
        ArrayList<Product> prods = new ArrayList<>();
        prods.add(new Product(1, "Nasi Goreng", 15000));
        prods.add(new Product(2, "Nasi Mawud", 20000));
        HashMap<String, Object> json = new HashMap();
        json.put("status", true);
        json.put("data", prods);
        String res = new Gson().toJson(json);
        response.setContentType("application/json");
        response.setCharacterEncoding("UTF-8");
        PrintWriter out = response.getWriter();
        out.print(res);
    }
}
```

Question?





Fakultas Informatika
School of Computing
Telkom University



THANK YOU